파일 기반 가계부
평가 답변 및 코드 해설

Money_menager | 평가 항목 1-4 전체 대응 | 2026-10-18

이 문서는 현재 저장소의 실제 구현을 기준으로 작성한 발표·질의응답 자료입니다. 각 질문에 대해 바로 말할 수 있는 답변, 파일과 줄 번호, 구현 과정, 명령 예시, 테스트 근거, 한계를 함께 정리했습니다. 소스코드는 수정하지 않았습니다.

읽는 순서

2-8쪽: 항목 1 - 기능, 영구 저장, 카테고리, 예산, CSV, 오류와 종료 코드
9-11쪽: 항목 2 - 모듈, 클래스, 안전한 수정·삭제
12-14쪽: 항목 3 - 제너레이터, 데코레이터, 타입 힌트
15-17쪽: 항목 4 - 저장 형식, 10만 건 확장, 깨진 CSV
18쪽: 평가 현장용 시연 순서 및 핵심 답변

검증 결과

2026-10-18에 기존 unittest 12개를 Python 3.14.7에서 실행했고 모두 통과했습니다. 명령: python -m unittest discover -s tests -v. 테스트에는 실제 CLI 하위 프로세스 호출, 재실행, CSV 왕복, 오류 종료, 원본 보존, 교체 실패, 1만 건 정렬이 포함됩니다. 10만 건 성능과 정적 타입 검사 실행은 확인하지 않았습니다.

예시 실행 전 확인

프로젝트 루트에서 Python 3.10 이상으로 실행합니다. 예시의 python은 사용자의 Python 실행 명령입니다. 이 환경에서는 Windows 별칭 대신 설치된 Python의 전체 경로로 검증했습니다. --data-dir는 서브명령 앞에 씁니다. ./eval-data는 새 연습 폴더로 사용하고, 샘플 import는 한 번만 실행하세요. 반복 import는 새 ID로 거래를 중복 추가합니다.

```
python -m budget_app --data-dir ./eval-data import --from examples/transactions.csv
python -m budget_app --data-dir ./eval-data list --limit 10
```

표기와 설명의 범위

파일:줄 번호는 작성 시점 소스를 가리킵니다. 명령 출력은 기존 테스트로 확인된 값 또는 계산에 따른 예상값이며, 모든 명령의 원문 실행 로그는 아닙니다. 성능 개선·오류 리포트 확대 등은 현재 구현과 구분해 제안으로 표시했습니다.

1-1. 기본 명령 8개는 어떻게 동작하나요?

평가 때 이렇게 답하세요

"CLI에서 명령을 해석하고 BudgetService로 처리를 넘깁니다. 거래는 검증 후 저장하고, 목록·검색은 날짜 내림차순으로 출력합니다. 요약은 월별 집계이며 수정·삭제는 ID로 대상을 찾습니다. CSV 입출력도 별도 명령으로 구현했습니다."

구현 위치

budget_app/cli.py:16-55 명령 정의, :67-124 실행 분기. service.py:44 select, :79 add, :90 change, :131 summary, :148 import_csv, :176 export_csv.

동작 원리와 구체적인 예

add는 날짜→종류→카테고리→금액→메모→태그를 대화형으로 받습니다. list 기본 한도는 10입니다. search는 기간·종류·카테고리·메모·태그 조건을 AND로 결합합니다. 메모는 대소문자 무시 부분 검색, 태그는 정확한 일치입니다. 같은 날짜는 나중에 저장된 거래가 먼저입니다. update는 전달한 필드만 수정하고 ID는 유지합니다. delete는 대상 행을 재작성 결과에서 제외합니다.

```
python -m budget_app --data-dir ./eval-data list --limit 2
python -m budget_app --data-dir ./eval-data search --category food --tag lunch
python -m budget_app --data-dir ./eval-data summary --month 2024-01
python -m budget_app --data-dir ./eval-data update --id 실제ID --amount 6000
python -m budget_app --data-dir ./eval-data delete --id 실제ID
```

확인 근거와 주의점

tests/test_budget.py:36 test_sample_summary_filters_and_order, :52 test_add_update_delete_and_restart, :63 test_csv_round_trip에서 검증합니다. 실제ID는 list에 출력되는 UUID로 바꿉니다. 수정·삭제 시연은 원본 작업 데이터 대신 연습 폴더에서 수행하세요.

샘플 4건에서 최신 2건 날짜는 2024-02-01, 2024-01-15입니다. 1월 요약은 수입 20,000원, 지출 6,500원, 잔액 13,500원입니다. 검색 조건이 없으면 전체 검색이며 결과가 없으면 "데이터 없음"을 출력합니다.

1-2. 재실행해도 데이터가 유지되나요?

평가 때 이렇게 답하세요

"거래·카테고리·월별 예산을 UTF-8 JSONL 파일 3개에 각각 저장합니다. 명령을 다시 실행할 때 같은 폴더의 파일을 읽으므로 프로세스가 종료되어도 데이터가 유지됩니다."

구현 위치

budget_app/repository.py:12-26 Repository 초기화와 paths, :28 records, :41 write, :56 category_names, :64 budget_values, :75 iter_transactions. cli.py:18 데이터 폴더 옵션.

동작 원리와 구체적인 예

기본 저장 경로는 현재 작업 폴더 기준 ./data입니다. transactions.jsonl에는 id/type/date/amount/category/memo/tags를 저장합니다. categories.jsonl에는 name을, budgets.jsonl에는 month/amount를 저장합니다. 폴더가 없으면 생성하며 파일은 touch(exist_ok=True)로 기존 내용을 지우지 않고 준비합니다. 카테고리가 비어 있으면 food/transport/salary/etc를 초기화합니다.

```
python -m budget_app --data-dir ./eval-data budget set --month 2024-01 --amount 5000
# 앞 명령이 종료된 뒤 새 프로세스로 확인
python -m budget_app --data-dir ./eval-data summary --month 2024-01
Get-ChildItem ./eval-data/*.jsonl
```

실제 코드 | budget_app/repository.py:13-22

```
13      def __init__(self, directory: str | Path):
14          self.directory = Path(directory)
15          self.transactions = self.directory / "transactions.jsonl"
16          self.categories = self.directory / "categories.jsonl"
17          self.budgets = self.directory / "budgets.jsonl"
18          self.directory.mkdir(parents=True, exist_ok=True)
19          for path in self.paths:
20              path.touch(exist_ok=True)
21          if not self.category_names():
22              self.write(self.categories, ({"name": name} for name in ("food",
"transport", "salary", "etc")))
```

확인 근거와 주의점

tests/test_budget.py:52는 CLI로 추가·수정·삭제한 결과와 새 Repository에서 읽은 상태를 확인합니다. :98은 카테고리와 예산을 새 Repository로 재조회합니다. :31 cli()는 매번 별도 Python 프로세스를 실행합니다.

같은 --data-dir와 같은 실행 기준 폴더를 사용해야 같은 데이터를 봅니다. 파일이 있다는 것만으로 복구·백업이 보장되지는 않습니다. JSONL 손상은 읽을 때 오류로 안내하며 자동 복구 기능은 없습니다.

1-3. 카테고리와 사용 중인 분류 삭제

평가 때 이렇게 답하세요

"이름을 검증해 추가·목록·삭제를 처리합니다. 이미 등록된 이름이나 없는 이름은 오류로 안내하고, 거래에서 사용 중인 카테고리는 삭제를 막아 참조가 깨지지 않도록 했습니다."

구현 위치

budget_app/service.py:109-124 category(), :75 validate_category(). repository.py:56 category_names(). models.py:37 category_name(). cli.py:100-107 카테고리 입출력.

동작 원리와 구체적인 예

추가·삭제의 이름은 명령 옵션이 아니라 input()으로 받습니다. 앞뒤 공백은 제거하고 빈 문자열이나 줄바꿈은 거부합니다. add는 set에 이름을 추가하고 정렬해 저장합니다. remove는 전체 거래를 검증하며 대상 이름의 사용 여부를 확인한 뒤, 사용되지 않을 때만 categories.jsonl을 교체합니다. 목록은 이름순입니다.

```
python -m budget_app --data-dir ./eval-data category add
# 카테고리 이름: books
python -m budget_app --data-dir ./eval-data category list
python -m budget_app --data-dir ./eval-data category remove
# 카테고리 이름: books -> 미사용이면 삭제
# 카테고리 이름: food -> 샘플 거래가 사용하므로 오류
```

확인 근거와 주의점

tests/test_budget.py:98에서 books 추가·재조회·삭제, 사용 중인 food 삭제 거부를 확인합니다. :166에서 category add/list/remove를 실제 CLI로 실행합니다.

food를 지우려면 해당 거래를 등록된 다른 분류로 update하거나 거래 자체를 delete한 다음 다시 삭제해야 합니다. 현재 구현은 자동 이관을 하지 않습니다. 사용 중 삭제 거부는 기능 누락이 아니라 데이터 참조를 유지하기 위한 선택입니다.

1-4. 예산 저장·사용률·초과 경고

평가 때 이렇게 답하세요

"예산은 YYYY-MM을 키로 저장하고 같은 월을 설정하면 갱신합니다. 요약에서 지출/예산×100을 사용률로 출력하며 지출이 예산보다 클 때 초과 경고를 보여줍니다."

구현 위치

budget_app/service.py:126-129 set_budget(), :131-146 summary(). repository.py:64-73 예산 읽기 검증. cli.py:84-99 출력과 budget 분기.

동작 원리와 구체적인 예

월은 실제 존재하는 YYYY-MM이어야 하고 금액은 1 이상 정수입니다. 예산을 0으로 설정할 수 없어 사용률 계산의 0 나눗셈을 방지합니다. 예산은 수입이나 잔액이 아닌 지출과 비교합니다. 지출과 예산이 같으면 100.0%이며 초과 경고는 없습니다. 예산이 없으면 사용률은 출력하지 않습니다.

```
python -m budget_app --data-dir ./eval-data budget set --month 2024-01 --amount 5000
python -m budget_app --data-dir ./eval-data summary --month 2024-01 --top 3
# 수입 20,000 / 지출 6,500 / 잔액 13,500
# 예산 5,000 / 사용률 130.0% / [경고] 예산 초과
# TOP: food 5,000, transport 1,500
```

실제 코드 | budget_app/cli.py:89-93

```
 89          print(f"총 수입: {result['income']:,}원₩n총 지출: {result['expense']:,}원₩n잔액:
{result['balance']:,}원")
 90          if result["budget"] is not None:
 91              print(f"예산: {result['budget']:,}원 (사용률 {result['expense'] /
result['budget'] * 100:.1f}%)")
 92              if result["expense"] > result["budget"]:
 93                  print("[경고] 예산 초과")
```

확인 근거와 주의점

tests/test_budget.py:36에서 합계·TOP·예산 값과 CLI의 "130.0%", "예산 초과"를 확인합니다. :98은 같은 월 100→5000 재설정 후 예산이 하나만 남는지 확인합니다.

거래가 없는 월은 "데이터 없음"과 0원 합계를 출력합니다. 그 월 예산이 있으면 사용률은 0.0%입니다. summary의 반환값은 집계용 dict이고 실제 사용률 계산과 경고 문구는 cli.py에서 담당합니다.

1-5. CSV 인코딩·헤더·컬럼 계약

평가 때 이렇게 답하세요

"내보내기는 UTF-8에 date,type,category,amount,memo,tags 헤더를 씁니다. 가져오기는 앞의 필수 4개 열을 검증하고 선택 열을 기본 빈 값으로 처리합니다. 쉼표·따옴표 처리는 csv 모듈에 맡겼습니다."

구현 위치

budget_app/service.py:16 FIELDS, :148-174 import_csv(), :176-202 export_csv(). models.py:79 create(). examples/transactions.csv:1-5 실제 샘플.

동작 원리와 구체적인 예

date는 실제 YYYY-MM-DD, type은 income/expense, category는 등록된 이름, amount는 양수 정수입니다. memo/tags는 생략할 수 있고 tags는 쉼표 구분 문자열을 내부 list[str]로 바꿉니다. 입력은 utf-8-sig로 BOM도 허용하며 출력은 BOM 없는 utf-8입니다. 내보내기는 월 또는 시작·종료일을 요구합니다. ID는 CSV에 내보내지 않고 import마다 새 UUID를 생성합니다.

```
date,type,category,amount,memo,tags
2024-01-15,expense,food,5000,"친구와 점심, 분식","meal,lunch"

python -m budget_app --data-dir ./eval-data export --out ./january.csv --month 2024-01
python -m budget_app --data-dir ./roundtrip-data import --from ./january.csv
```

확인 근거와 주의점

tests/test_budget.py:63은 CSV 왕복 후 합계·쉼표 포함 메모·태그를 확인합니다. :166은 헤더만 있으면 0건 성공, 필수 헤더가 없으면 실패함을 검증합니다. :135는 원본 JSONL로 export하는 것을 차단하는지 확인합니다.

현재 import는 열 순서 변경을 허용하며 중복 헤더와 행 열 개수 불일치는 거부합니다. 알 수 없는 추가 헤더는 무시합니다. 따라서 "정확히 6개 열만 허용한다"고 설명하면 안 됩니다. 동일 CSV 재가져오기의 중복 방지와 기존 ID 보존은 지원하지 않습니다.

1-6. 오류 원인과 해결 힌트

평가 때 이렇게 답하세요

"업무 오류는 AppError에 원인과 힌트를 담습니다. CLI 진입점에 적용한 데코레이터가 입력·파일 오류를 표준 오류로 출력하고 실패를 반환하므로 일반적인 잘못된 사용에서 스택트레이스를 보이지 않습니다."

구현 위치

budget_app/errors.py:7-10 AppError, :13-26 handle_errors. cli.py:11-13 Parser.error(), :67 @handle_errors. models.py:11-40 입력 검증.

동작 원리와 구체적인 예

날짜 형식·양수 금액·등록 카테고리·없는 ID를 업무 계층에서 검사합니다. argparse의 기본 오류도 Parser.error에서 AppError로 바꿔 같은 출력 형식을 씁니다. OSError/UnicodeError는 경로·권한·UTF-8 확인을 안내합니다. EOFError/KeyboardInterrupt는 입력 취소로 안내합니다. JSONL 읽기 오류에는 파일과 줄, CSV 행 오류에는 reader.line_num이 포함됩니다.

```
python -m budget_app --data-dir ./eval-data list --limit 0
# [오류] 양수 정수가 필요합니다: 0
# [힌트] 쉼표 없이 1 이상의 정수를 입력하세요.
python -m budget_app --data-dir ./eval-data import --from ./missing.csv
# [오류] 파일 처리 실패: ...
# [힌트] 경로·권한과 UTF-8 인코딩을 확인하세요.
```

확인 근거와 주의점

tests/test_budget.py:120은 잘못된 limit/월/없는 ID/기간/옵션/TOP 입력에 대해 비정상 종료, [힌트] 존재, Traceback 부재를 확인합니다. :84는 손상 JSONL에서 변경이 실패하고 원본이 유지되는지 확인합니다.

모든 예외를 무조건 숨기지는 않습니다. 예상하지 못한 프로그래밍 오류는 처리 범위 밖입니다. 파일 오류를 "파일 없음만 처리한다"거나 "어떤 상황에서도 traceback이 없다"고 과장하지 않는 것이 정확합니다.

1-7. 오류 종료 코드 확인

평가 때 이렇게 답하세요

"main은 정상 처리 후 0을 반환하고 오류 데코레이터는 1을 반환합니다. __main__.py가 그 값을 SystemExit에 전달하므로 운영체제에서 성공과 실패를 구분할 수 있습니다."

구현 위치

budget_app/cli.py:124 return 0. errors.py:24 return 1. __main__.py:1-3 main 호출 및 SystemExit. tests/test_budget.py:120-133 오류와 도움말 검증.

동작 원리와 구체적인 예

출력 문구와 종료 코드는 별개입니다. 메시지를 출력하기만 하고 0으로 종료하면 자동화 도구는 성공으로 오인합니다. PowerShell에서는 외부 명령 직후 $LASTEXITCODE를 읽습니다. 다음 명령이 실행되기 전에 저장해야 해당 명령의 값을 확실히 확인할 수 있습니다. --help의 정상 출력은 0입니다.

```
python -m budget_app --data-dir ./eval-data delete --id missing
$failureCode = $LASTEXITCODE
Write-Output $failureCode
# 예상: 1
python -m budget_app --data-dir ./eval-data list --limit 1
Write-Output $LASTEXITCODE
# 정상 저장소라면 예상: 0
```

실제 코드 | budget_app/errors.py:13-25

```
 13  def handle_errors(function: Callable) -> Callable:
 14      @wraps(function)
 15      def wrapper(*args, **kwargs) -> int:
 16          try:
 17              return function(*args, **kwargs)
 18          except AppError as error:
 19              print(f"[오류] {error}₩n[힌트] {error.hint}", file=sys.stderr)
 20          except (OSError, UnicodeError) as error:
 21              print(f"[오류] 파일 처리 실패: {error}₩n[힌트] 경로·권한과 UTF-8 인코딩을 확인하세요.",
file=sys.stderr)
 22          except (EOFError, KeyboardInterrupt):
 23              print("[오류] 입력이 취소되었습니다.₩n[힌트] 명령을 다시 실행하세요.", file=sys.stderr)
 24          return 1
 25      return wrapper
```

확인 근거와 주의점

tests/test_budget.py:31 cli()는 subprocess.run의 returncode를 수집합니다. :120 오류 케이스에서 assertNotEqual(returncode, 0)을, 도움말에서는 assertEqual(returncode, 0)을 수행합니다.

평가 시 "터미널에 오류가 떴다"뿐 아니라 "종료 코드가 1이다"를 같이 보여주세요. 배치 작업은 이 코드로 다음 작업 중단·재시도·실패 표시를 결정할 수 있습니다.

2-1. 모듈의 책임을 어떻게 나눴나요?

평가 때 이렇게 답하세요

"사용자 입출력, 업무 규칙, 저장 방식, 데이터 유효성, 공통 오류 처리를 분리했습니다. 변경 이유가 다른 코드를 나누어 화면 형식을 바꿀 때 저장 코드를 건드리지 않도록 설계했습니다."

구현 위치

budget_app/cli.py:16/58/68 입력·출력·분기. service.py:19 BudgetService. repository.py:12 Repository. models.py:44 Transaction. errors.py:7/13 오류. __main__.py:1-3 실행 진입점.

동작 원리와 구체적인 예

cli.py는 옵션과 input/print를 담당합니다. service.py는 검색·집계·카테고리 참조·CSV 규칙을 조합합니다. repository.py는 파일 경로, JSONL 읽기·쓰기와 원자적 교체를 담당합니다. models.py는 날짜·금액·필드 검증과 데이터 변환을 담당합니다. errors.py는 사용자용 오류 표현과 공통 예외 처리를 담당합니다. 실행용 __main__.py까지 6개 책임 단위입니다.

```
호출 흐름 예: update
__main__.py -> cli.main()
-> BudgetService.change(id, changes)
-> Transaction.from_dict() / validate_category()
-> Repository.write() -> 임시 파일 교체
```

확인 근거와 주의점

tests/test_budget.py는 서비스 단위 검증과 실제 CLI 검증을 함께 수행합니다. 예를 들어 :63 CSV 왕복은 CLI 출력과 무관하게 서비스를 호출하고, :52는 프로세스 경계까지 검증합니다.

판단 기준은 파일 수 자체가 아니라 변경 이유입니다. 출력 형식을 바꾸면 cli.py, 저장 방식을 바꾸면 주로 repository.py, 새로운 거래 검증 규칙은 models.py를 수정합니다. 다만 service.py가 임시 SQLite와 CSV I/O도 담당하므로 완전한 저장소 추상화는 아니며, 규모가 커지면 정렬·CSV 어댑터 분리를 고려할 수 있습니다.

2-2. 클래스 책임 경계는 어떻게 정했나요?

평가 때 이렇게 답하세요

"Transaction은 거래 한 건의 유효성, Repository는 저장과 복원, BudgetService는 여러 거래와 카테고리·예산을 연결하는 업무 흐름을 맡겼습니다. 데이터 한 건의 규칙과 저장소 전체의 규칙을 구분했습니다."

구현 위치

budget_app/models.py:43-81 Transaction. repository.py:12-84 Repository. service.py:19-202 BudgetService. errors.py:7 AppError, cli.py:11 Parser는 보조 클래스입니다.

동작 원리와 구체적인 예

Transaction은 amount가 int이며 양수인지, type/date/memo/tags가 유효한지 검증합니다. 그러나 카테고리가 저장소에 등록됐는지는 파일 접근이 필요하므로 BudgetService.validate_category가 확인합니다. Repository는 UTF-8 파일을 순회·검증하고 교체합니다. BudgetService는 add/change/summary/import/export와 같은 사용 목적을 실행합니다.

```
tx = Transaction.create(type="expense", date="2024-01-15",
                        amount="5000", category="food")
repo = Repository("./class-demo-data")
service = BudgetService(repo)
service.add(tx)
# 모델: 필드 검증 / 서비스: 등록 분류·ID 검사 / 저장소: 기록
```

확인 근거와 주의점

tests/test_budget.py:109는 Transaction이 bool/float/문자열/0 등 잘못된 저장 금액을 거부하는지 확인합니다. : 98은 서비스의 사용 중 카테고리 삭제 규칙과 저장소의 재조회 동작을 확인합니다.

의존성은 서비스가 Repository를 생성하지 않고 생성자에서 받는 방식입니다. 테스트는 임시 폴더의 Repository를 주입하므로 실제 사용자 데이터와 분리할 수 있습니다. 모델을 만들었다고 자동 저장되지는 않으며, 저장 성공은 service.add와 Repository.write가 완료돼야 성립합니다.

2-3. 파일 수정·삭제를 어떻게 안전하게 하나요?

평가 때 이렇게 답하세요

"원본을 직접 덮어쓰지 않고 같은 폴더의 임시 파일에 새 내용을 모두 씁니다. 검증과 flush/fsync가 끝난 다음 os.replace로 교체합니다. 중간에 실패하면 교체하지 않아 원본을 보존합니다."

구현 위치

budget_app/service.py:90-107 change(). repository.py:41-54 write(). tests/test_budget.py:84 실패 변경 원본 보존, :145 교체 실패 원본 보존.

동작 원리와 구체적인 예

change는 먼저 전체 거래를 임시 SQLite에 읽고 ID 중복·저장 데이터 오류를 확인합니다. 대상 ID가 없거나 changes가 빈 dict이면 실패합니다. 수정은 대상 row에 변경 값을 합친 뒤 Transaction으로 다시 검증하고, 삭제는 그 행을 yield하지 않습니다. 나머지는 원래 저장 순서로 새 파일에 기록합니다. 예외가 나면 finally에서 임시 파일을 정리합니다.

```
# 개념 흐름
원본 JSONL -> 전체 검증 및 대상 ID 확인
-> 새 행 생성기 -> 같은 폴더 임시 파일
-> flush -> fsync -> 파일 닫기 -> os.replace
# 실패가 교체 전에 발생하면 원본은 그대로 유지
```

실제 코드 | budget_app/repository.py:45-54

```
45                                  prefix=".budget-", delete=False) as stream:
46              temporary = Path(stream.name)
47              for row in rows:
48                  stream.write(json.dumps(row, ensure_ascii=False) + "₩n")
49              stream.flush()
50              os.fsync(stream.fileno())
51          os.replace(temporary, path)
52      finally:
53          if temporary is not None:
54              temporary.unlink(missing_ok=True)
```

확인 근거와 주의점

test_failed_mutations_preserve_bytes는 없는 ID, 잘못된 날짜·분류, 빈 변경, 손상 원본을 검증합니다. test_replace_failure_preserves_original은 os.replace에 PermissionError를 강제로 발생시켜 원본 바이트와 임시 파일 정리를 확인합니다.

보장 범위는 단일 파일 교체와 단일 프로세스 사용입니다. 파일 잠금이 없으므로 동시 실행 시 갱신 손실이 가능하고, 여러 파일 전체에 대한 트랜잭션은 없습니다. fsync는 파일 내용 동기화이며 모든 OS·파일시스템의 전원 장애 복구까지 완벽히 보장한다고 말하면 안 됩니다.

3-1. 제너레이터 스트리밍의 구현과 이점

평가 때 이렇게 답하세요

"JSONL을 한 줄씩 읽는 records와 거래로 변환하는 iter_transactions, 조건에 맞는 거래를 내보내는 select에 yield를 사용했습니다. CLI도 iterable을 바로 순회하여 전체 거래를 Python 리스트에 쌓지 않습니다."

구현 위치

budget_app/repository.py:28-39 records(), :75-84 iter_transactions(). service.py:44-73 select(), :23-42 ordered(). cli.py:58-64 show_transactions().

동작 원리와 구체적인 예

records는 파일을 with로 열고 한 줄씩 JSON 객체를 yield합니다. iter_transactions는 모델과 카테고리를 검증해 거래를 yield합니다. select는 정렬 커서에서 한 건씩 복원하고 AND 조건을 검사해 yield합니다. 호출자는 next나 for로 필요할 때 결과를 받습니다. list의 limit는 출력 건수를 제한합니다.

```
transactions = service.select(category="food", limit=2)
for tx in transactions:
    print(tx.date, tx.amount)
# list(service.select())처럼 전체를 리스트로 만들지 않아도 됨
# 직접 next() 후 중단한다면 transactions.close()로 자원 정리
```

실제 코드 | budget_app/repository.py:28-39

```
28      def records(self, path: Path) -> Iterator[dict[str, Any]]:
29          with path.open(encoding="utf-8") as stream:
30              for number, line in enumerate(stream, 1):
31                  if not line.strip():
32                      continue
33                  try:
34                      row = json.loads(line)
35                      if not isinstance(row, dict):
36                          raise ValueError("JSON 객체가 아닙니다")
37                  except ValueError as error:
38                      raise AppError(f"{path} {number}행 오류: {error}", "원본을 백업하고 해당 행을
복구하세요.") from None
39                  yield row
```

확인 근거와 주의점

tests/test_budget.py:155는 순서가 섞인 1만 건을 기록해 최신 3건 ID, 제너레이터 조기 close(), 합계 10,000을 검증합니다. 이는 10만 건 실측이나 상수 메모리 사용량 측정은 아닙니다.

핵심 한계: ordered는 첫 yield 전에 모든 JSONL 거래를 임시 디스크 SQLite에 적재하고 정렬합니다. 따라서 limit=1도 원본 전체를 읽습니다. Python 거래 리스트를 피하는 이점은 있지만, 첫 응답 지연과 임시 디스크 사용은 O(N) 규모입니다. SQLite 캐시·정렬 버퍼와 카테고리 집합도 있으므로 "메모리가 무조건 O(1)"이라고 단정하지 않습니다.

3-2. 데코레이터로 분리한 공통 기능

평가 때 이렇게 답하세요

"모든 명령에서 반복되는 예외 처리와 오류 출력 규칙을 handle_errors 데코레이터로 분리했습니다. 업무 코드는 검증 오류를 발생시키고, CLI 경계에서 사용자 메시지와 종료 코드로 바꾸도록 했습니다."

구현 위치

budget_app/errors.py:13-25 handle_errors(). budget_app/cli.py:67-68 실제 적용. service.py:23 @contextmanager, models.py:43 @dataclass도 사용하지만 과제의 직접 구현 공통 기능은 handle_errors입니다.

동작 원리와 구체적인 예

@handle_errors는 개념적으로 main = handle_errors(main)입니다. wrapper가 실제 main을 호출하고 AppError이면 원인·힌트를 stderr에 출력합니다. 파일·인코딩 오류와 사용자 취소도 일관되게 처리합니다.
@wraps(function)는 원래 함수명과 문서 문자열 같은 메타데이터를 유지해 디버깅과 도구 사용에 도움이 됩니다.

```
@handle_errors
def main(argv: list[str] | None = None) -> int:
    ...  # 명령 분기와 업무 호출
    return 0

# 서비스는 print 대신 예외 전달
raise AppError("없는 데이터: missing", "list에서 ID를 확인하세요.")
```

확인 근거와 주의점

tests/test_budget.py:120의 여러 명령이 같은 [힌트] 규칙과 비정상 종료를 보이는 것이 실제 적용 근거입니다. main이 반환한 오류 값은 __main__.py에서 프로세스 종료 코드로 전달됩니다.

왜 필요한가: 명령마다 try/except를 복사하면 힌트나 종료 코드 정책이 달라지기 쉽습니다. 한 곳에서 수정하면 모든 명령에 적용됩니다. 로그·성능 측정 데코레이터는 현재 구현하지 않았습니다. contextmanager는 정렬 DB 자원 수명 관리이며 직접 구현한 오류 처리 데코레이터와 목적이 다릅니다.

3-3. 타입 힌트의 이점과 실제 검증

평가 때 이렇게 답하세요

"금액은 int, 태그는 list[str], 조회 결과는 Iterator[Transaction]처럼 계약을 명시했습니다. IDE와 타입 검사기가 잘못된 사용을 찾기 쉬워지고, 협업자가 반환값과 입력 구조를 바로 이해할 수 있습니다."

구현 위치

budget_app/models.py:44-51 필드 타입, :27 positive(str | int) -> int, :68 to_dict(), :73 from_dict(). repository.py:28/41 입출력 계약. service.py:44-47 select(). cli.py:58 Iterable[Transaction].

동작 원리와 구체적인 예

positive는 CLI 문자열이나 정수를 받아 검증 후 int로 반환하므로 금액 계산 단계가 단순해집니다. Iterator[Transaction]은 전체 list가 아닌 순차 소비 결과임을 표현합니다. Iterable[Transaction]을 받는 출력 함수는 리스트와 제너레이터 모두 처리할 수 있습니다. str | None은 검색 조건을 지정하지 않은 상태도 명시합니다.

```
# 실제 모델 계약
amount: int
tags: list[str]

# 잘못된 사용 예: 정적 검사 도구가 있다면 진단 가능
# tx.amount = "5000"
# tx.tags = "meal,lunch"
# 올바른 값: tx.amount = 5000 / tx.tags = ["meal", "lunch"]
```

실제 코드 | budget_app/models.py:44-51

```
44  class Transaction:
45      id: str
46      type: str
47      date: str
48      amount: int
49      category: str
50      memo: str = ""
51      tags: list[str] = field(default_factory=list)
```

확인 근거와 주의점

현재 저장소에서 mypy/pyright 실행 근거는 확인하지 않았습니다. 타입 힌트 자체는 런타임에 타입을 강제하지 않습니다. 실제 검증 근거는 models.py:53-66의 __post_init__와 tests/test_budget.py:109의 잘못된 저장 금액 거부 테스트입니다.

Transaction.from_dict에 amount="10"을 넣으면 힌트 때문이 아니라 type(self.amount) is not int 검사 때문에 AppError가 납니다. bool은 int의 하위 타입이지만 저장 금액으로 거부합니다. 개선 여지로 summary -> dict를 TypedDict로 구체화하고 Callable에도 인자·반환 타입을 명시할 수 있습니다.

4-1. JSONL과 CSV 비교 및 선택 근거

평가 때 이렇게 답하세요

"내부 저장은 중첩 태그와 정수 타입을 자연스럽게 표현하고 한 줄씩 검증하기 쉬운 JSONL을 선택했습니다. 외부 교환은 엑셀 등에서 다루기 편한 CSV로 제공합니다."

구현 위치

budget_app/repository.py:15-17 내부 파일 확장자, :28 JSON 파싱, :41 JSON 직렬화. service.py:16/148/176 CSV 계약과 변환. models.py:51 tags 리스트.

동작 원리와 구체적인 예

JSONL 장점: 거래 한 건이 객체 한 줄이라 필드명을 함께 보며 복구할 수 있고 tags 배열과 amount 숫자를 표현하기 쉽습니다. 단점: 키 반복으로 크기가 커지고 엑셀에서 바로 열기 어렵습니다. CSV 장점: 표 형태로 작고 도구 호환성이 좋습니다. 단점: 모든 열의 날짜·금액 타입을 다시 해석해야 하며 쉼표·따옴표·줄바꿈 인용 처리가 필요하고 태그 리스트 표현도 별도 규칙이 필요합니다.

```
# 내부 JSONL의 개념 예
{"id":"example-id","type":"expense","date":"2024-01-15",
 "amount":5000,"category":"food","memo":"점심","tags":["meal","lunch"]}
# 실제 JSONL 저장에서는 위 객체 전체가 한 줄
# CSV 교환 예
date,type,category,amount,memo,tags
2024-01-15,expense,food,5000,점심,"meal,lunch"
```

확인 근거와 주의점

tests/test_budget.py:63이 외부 CSV 변환 후 메모·태그·집계 보존을 검증합니다. JSONL을 택했다고 자동으로 데이터 타입이 안전해지는 것은 아니며 Transaction의 필드 검증이 함께 필요합니다.

선택의 근거는 내부 객체 구조와 줄 단위 순회입니다. JSONL도 임의 ID 검색 인덱스나 안전한 행내 수정은 제공하지 않습니다. 현재 add/update/delete가 전체 재작성인 것은 실제 구현의 선택이며, JSONL 형식 자체가 항상 전체 재작성을 요구하는 것은 아닙니다. 임시 SQLite는 정렬용이고 영구 저장 형식은 JSONL입니다.

4-2. 거래 10만 건의 병목과 개선

평가 때 이렇게 답하세요

"현재는 조회마다 전체 JSONL을 읽어 임시 SQLite를 새로 만들고 정렬하므로 첫 응답과 디스크 I/O가 병목입니다. 추가·수정·삭제도 전체 거래를 검증·재작성합니다. 조회 인덱스를 유지하거나 영구 DB로 전환해 개선할 수 있습니다."

구현 위치

budget_app/service.py:24-42 ordered() 전체 적재, :61-73 정렬 후 필터, :79-107 변경, :131 summary(). repository.py:41 write(). tests/test_budget.py:155는 1만 건 검증입니다.

동작 원리와 구체적인 예

매 조회는 N건 JSON 파싱·검증·직렬화·임시 DB 삽입을 수행합니다. 날짜 정렬에 적합한 인덱스가 없어 정렬 비용은 일반적으로 O(N log N) 수준이며 필터는 SQL WHERE가 아니라 정렬 후 Python에서 검사합니다. limit는 원본 읽기를 줄이지 못합니다. summary도 정렬을 거쳐 불필요한 작업을 합니다. 카테고리 집계는 분류 수만큼 메모리를 씁니다.

개선 우선순위 예
1. summary: 날짜 정렬 없이 JSONL 1회 순회로 합계 계산
2. 임시 정렬: WHERE 조건을 SQL로 옮기고 적합한 인덱스 검토
3. 파일 조건 유지: 파생 인덱스·월별 분할·캐시 무효화 설계
4. 조건 변경 가능: 영구 SQLite + 트랜잭션 + 날짜/분류 인덱스
5. 동일 데이터로 첫 응답, 전체 시간, RSS, 임시 디스크를 비교

확인 근거와 주의점

이번 검증은 1만 건 테스트 통과까지입니다. 10만 건의 실행 시간·최대 메모리는 측정하지 않아 숫자를 제시하지 않습니다. 제너레이터라는 이유만으로 대용량 성능을 보장할 수 없습니다.

10만 건에서 한 건씩 add하면 매번 재작성하므로 누적 작업량은 $O(N^2)$ 규모가 될 수 있습니다. 일괄 import가 반복 add보다 유리합니다. 영구 SQLite는 과제의 JSONL/CSV 저장 조건과 달라질 수 있으므로 제출용에서는 조건을 유지한 개선과 서비스 확장안을 나눠 설명해야 합니다. 추가 인덱스도 검색 조건·정렬 방향에 맞춰 실측해야 합니다.

4-3. 깨진 CSV 행과 사용자 신뢰

평가 때 이렇게 답하세요

"현재는 한 행이라도 잘못되면 가져오기를 전체 실패로 처리합니다. 정상 행을 먼저 읽었더라도 임시 파일만 수정한 상태이므로 원본 거래는 유지됩니다. 오류 행 번호와 원인을 보여주고 수정 후 다시 가져오도록 안내합니다."

구현 위치

budget_app/service.py:148-174 import_csv(), 특히 :152 rows(), :161-172 행 검증. repository.py:41-54 임시 파일 기록과 최종 교체. tests/test_budget.py:74 원본 바이트 보존.

동작 원리와 구체적인 예

기존 거래를 임시 출력에 복사한 뒤 CSV를 순차 검증해 덧붙입니다. 첫 정상 행으로 count가 올라가도 아직 원본에는 반영되지 않습니다. 다음 행이 금액 -1이면 AppError가 발생해 os.replace에 도달하지 않고 임시 파일을 정리합니다. imported=N은 import_csv가 정상 반환한 뒤 CLI에서만 출력되므로 실패 중간 건수를 성공으로 알리지 않습니다.

```
date,type,category,amount
2024-01-01,income,salary,10
2024-01-01,expense,food,-1

# 예상: CSV 3행 금액 오류 + 수정 후 재시도 힌트
# 결과: 첫 정상 행도 추가되지 않음 / 원본 거래 그대로
# 실패 종료 코드: 1
```

확인 근거와 주의점

tests/test_budget.py:74는 정상 행 다음에 깨진 행을 넣고 import 실패 후 원본 read_bytes()가 완전히 같은지, .budget-* 임시 파일이 남지 않는지 확인합니다. 이는 실패 후 되돌려 쓰는 방식보다 "교체 전 미반영"에 가깝습니다.

현재는 첫 오류만 보고하며 전체 오류 목록 파일·부분 성공 옵션은 없습니다. 제안: 대량 입력은 사전 검사 모드와 행/컬럼/원인 리포트를 제공하고, 부분 성공 모드를 추가한다면 사용자 선택·성공/실패 건수·실패 행 CSV·중복 재시도 정책을 함께 제공해야 합니다. reader.line_num은 CSV 레코드 번호가 아닌 물리적 줄 번호라 여러 줄 메모에서는 차이가 납니다.

평가 현장용 시연 순서와 답변 요약

평가 때 이렇게 답하세요

"기능 시연 후 저장 파일과 구현 함수를 열어 설명하겠습니다. 실패 사례도 함께 보여 데이터가 손상되지 않고 종료 코드로 실패가 전달되는 것을 확인하겠습니다."

구현 위치

아래 순서는 새 ./eval-demo 폴더 사용을 가정합니다. 기존 데이터 폴더를 덮어쓰지 말고, 동일 이름의 연습 폴더가 있으면 새 이름을 선택하세요.

동작 원리와 구체적인 예

① 샘플을 한 번 import해 4건을 준비합니다. ② list/search에서 날짜순과 조건을 확인합니다. ③ 예산 5,000원과 1월 summary로 130.0%를 확인합니다. ④ 프로그램을 새로 실행해 지속성을 확인합니다. ⑤ export한 CSV의 헤더·메모·태그를 확인합니다. ⑥ category에서 미사용 삭제 성공과 사용 중 삭제 실패를 비교합니다. ⑦ 실제 ID로 update/delete 후 재조회합니다. ⑧ 오류 종료와 기존 테스트를 보여줍니다.

```
python -m budget_app --data-dir ./eval-demo import --from examples/transactions.csv
python -m budget_app --data-dir ./eval-demo list --limit 2
python -m budget_app --data-dir ./eval-demo search --category food --tag lunch
python -m budget_app --data-dir ./eval-demo budget set --month 2024-01 --amount 5000
python -m budget_app --data-dir ./eval-demo summary --month 2024-01
python -m budget_app --data-dir ./eval-demo export --out ./demo.csv --month 2024-01
python -m budget_app --data-dir ./eval-demo delete --id missing
Write-Output $LASTEXITCODE
python -m unittest discover -s tests -v
```

확인 근거와 주의점

발표 핵심: "저장은 Repository, 업무는 BudgetService, 한 건 검증은 Transaction입니다. 수정·삭제·import는 임시 파일을 완성한 뒤 교체합니다. 제너레이터는 전체 Python 리스트를 피하지만 정렬 전 전체 적재는 남아 있습니다."

추가 질문 대비: JSONL은 내부 구조 보존, CSV는 외부 교환에 유리합니다. 손상 import는 전체 실패로 기존 거래를 지킵니다. 타입 힌트는 설명·정적 분석용이고 실제 강제는 검증 코드가 합니다. 동시 실행 잠금·전체 오류 리포트·10만 건 실측은 현재 구현·검증 범위에 없습니다.